

From PyTorch to the NPU: LLM-Agent-Driven Model Conversion Across Heterogeneous Inference Runtimes

AI for Porting and Inferencing on OpenVINO, RKNN, TensorRT, and ONNX

Jianhao Su¹, Zhanwei Wu¹, Chia-Heng Tu², and ShengTing Huang¹

¹Qualcomm Technologies, Inc.

²National Cheng Kung University, Tainan, Taiwan

ABSTRACT

Edge AI model deployment is a multi-stage engineering process involving model conversion, operator compatibility handling, runtime integration, and precision verification. While prior work has demonstrated agent-based automation for Qualcomm AI Runtime, the broader edge inference runtime ecosystem, including Intel OpenVINO, Rockchip RKNN, NVIDIA TensorRT, and ONNX Runtime, presents distinct toolchains and optimization strategies.

This paper extends AIPC (AI Porting Conversion, an LLM agent-driven methodology for AI model deployment automation previously demonstrated on Qualcomm AI Runtime) to multi-runtime scenarios, proposing an LLM agent-driven approach for automated single-model-to-single-runtime deployment across heterogeneous inference backends, such as Intel OpenVINO, Rockchip RKNN, NVIDIA TensorRT, and ONNX Runtime. We decompose the edge AI deployment into standardized, verifiable stages, and inject runtime-specific domain knowledge into the agent execution flow through agent skills, auxiliary scripts, and staged verification loops.

Using representative vision models, we demonstrate that agent-based deployment can complete the conversion from a PyTorch model to its executable inference, targeting OpenVINO for x86/NPU, RKNN for RK3588, TensorRT for NVIDIA GPU, and ONNX Runtime for Qualcomm NPU with a focus on FP16 precision deployment feasibility verification. The contributions of this paper primarily lie in providing multi-runtime deployment engineering practice experience, toolchain mapping analysis, a layout-adaptation and inference-replacement layer that removes manual transpose insertion from the agent’s repair burden, and an empirical characterization of agent deviation behavior under structured knowledge injection, rather than large-scale systematic benchmarking or cross-runtime operator repair strategy comparison.

KEYWORDS

AI Model Deployment, Edge AI, Model Conversion, Multi-Runtime Porting, Large Language Model Agents, Automated Software Engineering

1 INTRODUCTION

As edge AI applications expand from image classification to object detection and multimodal understanding, model deployment has become a critical engineering stage determining whether a system can take advantage of underlying hardware. The deployment environment is further complicated by the diversity of target hardware and its runtime software, and each impose different constraints on model structure, operator support, quantization schemes, and

runtime APIs. Significant gaps remain between models and their associated code on these heterogeneous runtimes.

In practice, deployment goes far beyond model export. An example of a typical model deployment flow requires ONNX conversion, operator substitution, dynamic shape resolution, computational graph repair, runtime-specific compilation, preprocessing and post-processing adaptation, and precision verification. For each runtime software, developers must understand not only the model itself but also toolchain behavior, operator support boundaries, layout constraints, and platform-specific differences.

Large Language Models (LLMs) have demonstrated powerful capabilities in code generation, automatic repair, task orchestration, and tool use. Building on the AIPC framework initially demonstrated for Qualcomm AI Runtime (QAIRT) [16], this paper investigates the following question: *given multiple deployment toolchains and verification workflows, can LLM agents effectively take over the repetitive engineering work in cross-heterogeneous-runtime AI model deployment, and execute bounded automatic repair when conversion failures or runtime exceptions occur?*

The key question is not whether “LLMs understand every hardware detail,” but rather: when domain knowledge is explicitly provided through agent skills, auxiliary functions, and verification rules, can the LLM serve as a task orchestrator and executor, transforming complex multi-runtime deployment workflows into repeatable, verifiable, and continuously improvable automated workflows?

The major contributions of this paper are summarized as follows.

- (1) **Multi-Runtime Deployment Design.** We generalize the AIPC agent methodology from a single runtime (QAIRT) to four heterogeneous inference backends, including Intel OpenVINO, Rockchip RKNN, NVIDIA TensorRT, and ONNX Runtime QNN (Qualcomm Neural Network), demonstrating the preliminary feasibility of the “skill + verification loop” design pattern across different toolchains.
- (2) **Cross-Runtime Deployment Pipeline Comparison.** We provide deployment pipeline mapping analysis for five runtimes (including QAIRT), comparing conversion modes, output formats, and toolchain characteristics across runtimes. This analysis lays the foundation for subsequent systematic comparative studies in the future.
- (3) **Inference Replacer for Layout Adaptation.** A drop-in replacement for ONNX Runtime is presented, called ONNXWrapper, to automatically load the corresponding hardware runtime model for precision comparison and to perform NCHW ↔ NHWC layout adaptation at the ONNX level. This

removes the need for per-model manual transpose insertion and eliminates a class of layout-induced conversion failures.

- (4) **Empirical Characterization of Agent Deviation.** Empirical experiments have been conducted to evaluate the deviation behaviors of LLM agents with and without structured domain-knowledge injection. Through a controlled comparison of deployments, we find that agents deviate from prescribed workflows under both conditions. A careful design of verification mechanisms is needed to address this issue.

The remainder of this paper is organized as follows. Section 2 describes the inference runtimes addressed in this work, related work on agent-driven deployment, and the cross-runtime challenges that motivate our approach. Section 3 presents the agentic deployment methodology and the “skill + verification loop” design pattern. Section 4 details the deployment pipeline, agent skill specifics, and repair actions for each target runtime. Section 5 reports two groups of case studies on representative vision models. Section 6 concludes this work.

2 BACKGROUND

This section presents the concepts relevant to this work. Section 2.1 introduces the common edge AI deployment pipeline. Sections 2.2 through 2.6 describe the toolchain characteristics and deployment features of different target runtimes. Section 2.7 covers related work on agent-driven deployment, and Section 2.8 discusses the cross-runtime challenges that motivate this study.

2.1 Hardware Inference Runtime Overview

Edge AI model deployment ultimately requires execution on specific hardware platforms, where different hardware architectures (CPU, GPU, and NPU) provide their inference runtimes optimized for their hardware characteristics. Especially, native hardware runtimes can fully leverage hardware parallel computing capabilities, memory architectures, and dedicated acceleration units. It achieves significant performance improvements and power consumption reductions.

However, these native hardware runtimes differ significantly in toolchains, operator support, quantization schemes, and software interfaces (APIs). In terms of precision support, different devices and runtimes have varying support for low-precision formats. Some devices support floating-point low-precision formats like FP16 and BF16, some only support integer quantization formats (e.g., INT8, INT4), and others support multiple precisions simultaneously. These hardware-level constraints directly impact deployment workflows, with Neural Processing Unit (NPU) devices being particularly restrictive, offering far less flexibility than CPUs and GPUs.

A typical edge AI model deployment processing flow (pipeline) is illustrated in Fig. 1. The main steps of this processing pipeline are introduced as follows. It should confirm that the original model runs correctly and generates reference outputs. It exports the model to ONNX [7] intermediate representation as a common starting point for all runtimes, most commonly from a PyTorch [11] source model. It uses target runtime conversion tools to generate deployable formats (quantization may be completed as a built-in step of conversion). It verifies that the converted model’s output shapes

and precision meet expectations. After deployment stabilizes (the converted model is available), further optimization, quantization, and performance analysis can be performed.

2.2 Intel OpenVINO

Intel OpenVINO (Open Visual Inference and Neural Network Optimization) [5] is Intel’s toolkit specifically designed for optimizing and deploying deep learning models on Intel hardware, supporting CPUs, integrated GPUs, and Neural Processing Units (NPUs). Its deployment characteristics are as follows.

- Operator support varies across CPU, GPU, and NPU plugins. The NPU has the most limited operator set.
- Dynamic shape support varies by devices. CPUs and GPUs natively support dynamic shapes, while NPUs have limited dynamic shape support requiring workarounds, such as padding under certain conditions. Official pre-compiled models could be provided as alternatives to overcome this limitation.
- NPU memory constraints may require model partitioning or subgraph offloading.
- Model format is IR (Intermediate Representation, .xml + .bin). Device-related model caching mechanisms are also available. For example, caching compiled models to disk could be used to reduce initialization time on subsequent loads.

2.3 Rockchip RKNN

Rockchip RKNN (Rockchip Neural Network) [13] is the inference SDK for Rockchip NPUs. Its deployment characteristics are as follows.

- Precision and operator support are strongly correlated with specific chip models; different RKNN chips (e.g., RV1106 vs. RK3566) have different operator sets and quantization capabilities, requiring closer attention to underlying hardware specifications during deployment.
- Operator support is relatively limited, requiring custom implementations or CPU fallback.
- Strict constraints on tensor shapes, layout (preferring NHWC), and quantization parameters.

2.4 NVIDIA TensorRT

NVIDIA TensorRT [9] is a high-performance deep learning inference optimizer and runtime for NVIDIA GPUs, supporting data center and edge platforms. TensorRT provides two main model deployment paths. (1) **ONNX → TensorRT** exports the model to ONNX format first, then optimizing and compiling via the TensorRT Builder. (2) **Torch-TensorRT** directly compiles PyTorch models into TensorRT-optimized subgraphs via the PyTorch integration package, without manual ONNX export, suitable for mixed deployment scenarios requiring partial PyTorch native execution. Its deployment characteristics are as follows.

- Serialized engines cannot be ported across chips or platforms. Recompilation for the target GPU architecture is required.
- Some ONNX operators are unsupported, requiring plugin development or computational graph rewriting.

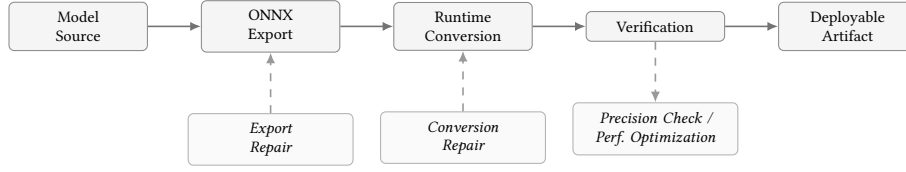


Fig. 1: A typical edge AI model deployment pipeline. Solid arrows show the forward flow; dashed arrows show the repair and verification paths that the agent exercises when a stage fails.

- Dynamic shape support has range limitations. TensorRT requires specifying minimum, optimal, and maximum values for each dynamic dimension in an Optimization Profile. Input shapes outside this range cannot be executed.
- Build-time optimization (strategy selection) may be time-consuming for complex models.

2.5 ONNX Runtime QNN

ONNX Runtime [8] is an ONNX inference engine supported by Microsoft. This runtime supports multiple hardware backends through the Execution Provider (EP) architecture. The QNN Execution Provider (QNN EP) integrates ONNX Runtime with QAIRT, enabling ONNX models to run on Qualcomm Hexagon NPUs across smartphones, notebooks, IoT, and other platforms. Its deployment characteristics are as follows.

- ONNX Runtime QNN EP operator coverage depends on QNN backend support. Unsupported operators fall back to CPU execution, potentially causing significant performance degradation.
- Compatibility between QAIRT SDK versions and ONNX Runtime versions requires careful management.
- QNN EP models require fixed input dimensions.

Olive model optimization tool. Microsoft Olive is a model optimization toolchain for ONNX Runtime, designed around the concept of **Recipes** (optimization passes). That is, users compose a series of optimization passes in declarative JSON configuration files as recipes for FP16 conversion, static quantization (INT8/INT16 calibration), graph-level operator fusion (OnnxModelOptimizer, OrtTransformersOptimization), QNN Context Binary generation, and runtime tuning (OrtPerfTuning), with full reproducibility and version traceability.

Instead of adopting the recipe pipeline mechanism, this work judiciously calls Olive’s QNNPreprocess pass, which is a preprocessing step designed for QNN EP compatibility. Especially, it automatically replaces or decomposes unsupported operators into equivalent supportable subgraphs, reducing the likelihood of operator fallback to CPU when the model executes on the HTP (Hexagon Tensor Processor) backend. This feature is a capability that `onnxruntime.quantization` alone cannot provide. Another consideration for adopting Olive is **future optimization flexibility**: when quantization calibration, graph optimization, or performance tuning needs arise later, passes can be directly added to the existing Olive calls without changing the toolchain or restructuring deployment scripts.

2.6 Qualcomm AI Runtime

QAIRT [12] is Qualcomm’s AI inference SDK, covering inference scenarios from lightweight vision models to generative large models. QAIRT primarily comprises two execution frameworks: SNPE and QNN. SNPE (Snapdragon Neural Processing Engine) uses DLC (Deep Learning Container) format with good cross-backend execution capability, suitable for rapid integration and prototype verification. QNN is closer to the hardware execution layer, achieving efficient deployment through platform-specific Context Binary generation, making it the preferred path for production environments. Its deployment characteristics are as follows.

- Precision and operator support are influenced by the target Snapdragon SoC model. Different chips’ NPU/HTP capabilities and operator sets differ.
- Context Binary generation of QNN is closely tied to the toolchain and system environment. For instance, for ARM64/Windows platforms, model conversion requires x64 emulation to complete compilation. For ARM/Linux platforms, the current version (v2.45) still lacks complete toolchain support, typically requiring development and compilation on x64/Linux hosts before deployment to target devices.
- Dynamic shapes are not supported on the QNN HTP backend. All input dimensions must be fixed before deployment.
- SDK version and target device compatibility require careful management.

Notably, while QAIRT SDK and ONNX Runtime QNN EP share the QNN library, they are two independent deployment paths. First, QAIRT SDK provides QNN as a standalone inference framework (ONNX → QNN Converter → Context Binary) without depending on ONNX Runtime. Second, ONNX Runtime QNN EP is an ONNX Runtime execution provider that achieves hardware acceleration by calling the QNN library, preserving ONNX Runtime’s inference pipeline and EP architecture. The difference lies in deployment pipeline orchestration and runtime integration level, not underlying hardware capabilities.

2.7 Related Work for AI Agent Deployment

This paper builds on the prior AIPC framework [16], which demonstrated the feasibility of LLM agent-driven QAIRT model deployment. Recently, the Huawei Ascend open-source community’s agent-skills repository [1] also adopted a similar “skill + verification” architecture, which provides the `atc-model-converter` skill module for Ascend NPU, covering the structured PyTorch → ONNX → .om (Offline Model) conversion workflow. The .om is Ascend NPU’s native inference format, compiled from ONNX

models by the Ascend Tensor Compiler, analogous to RKNN’s .rknn or TensorRT’s .engine. Their shared practice validates the “skill + verification loop” as a general design pattern for LLM agent-driven deployment.

In related work on agent-based hardware deployment, most research focuses on **subsystem optimization**, particularly operator and kernel-level performance improvements, including AccelOpt [18], KForge [14], Xe-Forge [15], EdgeFM [4], and AVO [2]. Li et al. [6] proposed a two-stage agent skill system for AMD XDNA 2 NPU, distilling human expertise into structured documentation for autonomous deployment of multiple LLMs.

Notably, **none of the above related work handles the complete end-to-end PyTorch → ONNX → target format model conversion with a deployment flow (pipeline) retargetable to various hardware platforms**. The aforementioned kernel optimization works focus on performance improvement of existing operators or kernels. Huawei Ascend’s agent-skills [1] is currently one of the few agent-driven works involving end-to-end conversion, but covers only a single runtime. AIPC’s differentiation lies in: systematizing the PyTorch → ONNX → multiple target formats conversion workflow, and exploring a unified deployment methodology across heterogeneous runtimes through operator compatibility handling, layout adaptation, and staged verification loops.

AIPC methodology further addresses two problems not explicitly solved in the above works. (1) **Unsupported operator handling** provides substitution, decomposition, and CPU fallback strategies for unsupported operators across runtimes. (2) **Tensor layout adaptation** automatically handles NCHW ↔ NHWC layout conversion at the ONNX level through ONNXWrapper.

2.8 Cross-Runtime Deployment Challenges and Motivation

Despite the variety of toolchains and APIs associated with different hardware runtimes discussed in Sections 2.2 to 2.6, these works commonly face four major challenges.

(1) **Operator Compatibility**. A subset of ONNX operators is supported by all these runtimes. Unsupported operators require equivalent substitution or computational graph rewriting before deployment. Furthermore, the set of unsupported operators differs across runtimes, and substitution strategies vary by runtime. In this case, it requires developers to master replacement knowledge on a per-platform basis.

(2) **Tensor Layout Mismatch**. Conflicts exist between PyTorch’s default NCHW layout and the NHWC layout preferred by some runtimes. Unaddressed layout mismatches cause conversion failures or silent performance degradation.

(3) **Dynamic vs. Fixed Shape Conflicts**. Research-oriented models often use dynamic input dimensions, while edge inference runtimes generally prefer fixed-shape compilation paths. Converting dynamic shapes to fixed shapes requires understanding model semantics and hardware constraints, and is one of the steps most frequently requiring manual intervention in deployment.

(4) **Quantization Dependence on Calibration Data and Environmental Complexity**. Quantization is not simply an instruction to execute. The representativeness of calibration data, consistency of preprocessing logic, and the complexity of environments with

coexisting multiple toolchain versions (e.g., Windows on ARM, x86 emulation) affect the stability of quantization results.

These common challenges mean that regardless of the target runtime, the deployment workflow involves substantial engineering work requiring domain knowledge, being repetitive and error-prone. Existing toolchains each address format conversion problems individually, but lack a deployment automation framework that can uniformly handle the above challenges across runtimes. This is the motivation for the LLM agent-driven methodology in this paper. This methodology explicitly encapsulates each runtime’s domain knowledge through skill modules, ensuring conversion quality through staged verification loops. Moreover, our proposed methodology enables these agents to automatically handle cross-platform common deployment challenges within constrained workflows.

3 AGENTIC DEPLOYMENT METHODOLOGY

In this work, an **agent** is an autonomous execution entity driven by Large Language Models with tool-calling capabilities. During the edge AI deployment process, the agent consists of three core components. (1) **LLM backend** provides code generation, error analysis, and task planning capabilities. (2) **Tool-calling interface** allows the agent to execute shell commands, read/write files, and invoke external toolchains. (3) **Skill modules** encapsulate runtime-specific domain knowledge (toolchain commands, operator compatibility tables, common failure patterns) in a structured manner to inject into the agent’s execution context. In subsequent sections, “agent,” “LLM agent,” and “AI agent” all refer to the same concept and are used interchangeably depending on context.

3.1 The “Skill + Verification Loop” Design Pattern

Section 2.1 outlines the staged pipeline for edge AI model deployment (Model Source → ONNX Export → Runtime Conversion → Verification → Deployable Artifact). Notably, the Runtime Conversion stage can be further divided into single-stage and dual-stage modes across different runtimes, distinguished by whether intermediate executable artifacts exist. **Single-stage conversion** directly generates the final deployable format from ONNX. That is, RKNN generates .rknn in one step via RKNN-Toolkit2, and TensorRT generates a serialized Engine (Plan file) in one step via trtexec.

Dual-stage conversion first produces an intermediate format, then further compiles it into device-specific executable artifacts. OpenVINO first generates device-agnostic IR (.xml + .bin) via ovc/mo, and its runtime calls the function compile_model() to compile the intermediate file into a device-specific model (with optional model cache). QAI RT first converts ONNX to QNN Model (.cpp/.bin) as an intermediate artifact, and then the intermediate files are compiled into a Context Binary (.bin). Additionally, ONNX Runtime QNN EP provides more flexibility. It can directly load ONNX models for execution via QNN EP (runtime compilation), or optionally enable QNN context binary to serialize compilation results into an ONNX model with EPContext nodes and a .bin file to accelerate subsequent loading.

The AIPC framework [16] decomposes this common pipeline into six verifiable stages, illustrated in Fig. 2. (1) **Model Preparation and Baseline Establishment** confirms the original model

runs correctly and establishes golden reference outputs. (2) **ONNX Conversion and Modularized Inference Pipeline** exports the model to ONNX intermediate representation as the common starting point for all runtimes. (3) **Runtime Model Construction** uses target runtime conversion tools to generate deployable formats. (4) **Inference Execution and Precision Alignment** verifies that the converted model’s output shapes, data types, and precision meet expectations. (5) **Optimization and Quantization (Optional)** enters optimization and quantization workflows only after the first four stages are stable, avoiding unnecessary uncertainty. (6) **Performance Analysis and Deployment Report** collects performance data and generates reusable deployment documentation.

The core of AIPC lies not in the common pipeline itself, but in the “skill + verification loop” mechanism wrapping the pipeline.

- (1) **Agent Skills** encapsulate runtime-specific knowledge: conversion commands, operator compatibility tables, quantization recipes, and common failure patterns with their fixes.
- (2) **Auxiliary Scripts** provide deterministic toolchain operations (model export, computational graph checking, inference benchmarking), which the agent invokes as tool calls.
- (3) **Verification Rules** define pass/fail criteria for each stage: ONNX model validity checks, conversion success/failure detection, output shape verification, and precision comparison with golden reference outputs.
- (4) **Failure Recoverability**. Model surgery, retry, and manual correction are integrated into the workflow, rather than treating failure as the end of the process. AIPC treats model surgery as a core capability of deployment automation rather than an exceptional case, with repair actions spanning three intervention levels. When ONNX export fails, modifying module implementations at the PyTorch source code level to eliminate operator or shape incompatibilities. When runtime conversion fails, patching nodes and attributes at the ONNX computational graph level. When runtime inference anomalies occur, performing further compatibility repairs based on actual errors, with fallback to earlier stages for reprocessing when necessary.
- (5) **ONNXWrapper Inference Replacement**. During the deployment verification stage, agents need to compare inference results against ONNX Runtime reference outputs for precision checking. **ONNXWrapper** is the inference replacement layer in AIPC, designed as a direct substitute for ONNX Runtime that additionally performs tensor layout adaptation. After replacing ONNX Runtime with ONNXWrapper in inference scripts, without modifying underlying code or regenerating target programs, the Wrapper automatically loads the corresponding hardware runtime model and produces output for comparison with ONNX Runtime results. This mechanism enables agents to rapidly complete multi-runtime precision verification without changing existing inference workflows. Its layout-adaptation role is detailed in Section 4.6.

3.2 Agent Skills

Table 1 lists the skill modules required by our proposed methodology.

Table 1: Skill modules required by the proposed methodology.

Skill Component	Description
Toolchain Reference	CLI commands, API signatures, configuration formats
Operator Compatibility Matrix	Supported/unsupported operators for each runtime version
Quantization Guide	Calibration procedures, precision trade-offs
Common Failure Patterns	Error-to-fix mappings for common conversion issues
Verification Scripts	Shape checkers, inference executors, precision comparators
Platform Constraints	Memory limits, shape constraints, layout requirements

3.3 Single-Model Single-Runtime Orchestration

The deployment scenario in this study is that given a PyTorch model and a target runtime (OpenVINO, RKNN, TensorRT, or ONNX Runtime QNN), the agent is responsible for handling the complete conversion workflow from the original model to executable inference on that runtime. The agent’s orchestration logic is as follows.

- (1) **Runtime Identification**. Based on the user-specified target runtime, the agent loads the corresponding skill modules, such as toolchain commands, operator compatibility table, common failure patterns.
- (2) **ONNX Export**. The agent exports the PyTorch model to ONNX intermediate representation as the common starting point for all runtimes.
- (3) **Runtime-Specific Conversion**. The agent invokes the corresponding conversion tool based on the target runtime (OpenVINO ovc, RKNN-Toolkit2, TensorRT trtexec, ONNX Runtime QNN EP), applying runtime-specific configuration parameters.
- (4) **Verification and Repair Loop**. The agent performs shape checking and inference precision verification on conversion results. If failures occur, the agent consults the repair patterns in the skill module to make corrections and re-verify.
- (5) **Deployable Artifact Output**. The agent generates model files that can be directly loaded and executed on the target runtime.

Each deployment is an independent task. The agent focuses only on a single model-to-single-runtime pairing, without cross-runtime parallel processing or shared repairs.

4 RUNTIME-SPECIFIC DEPLOYMENT PIPELINES

This section primarily explains that porting involves two parts, runtime-specific code and ONNXWrapper. To validate the AIPC methodology, this study ports to a single runtime at a time rather than simultaneously targeting multiple targets. In implementation, our basic approach is to first use AI assistance to adapt the AIPC Skill applicable to QAIPT, then through manual verification and experimental adjustment, until the required functional modules can complete testing.

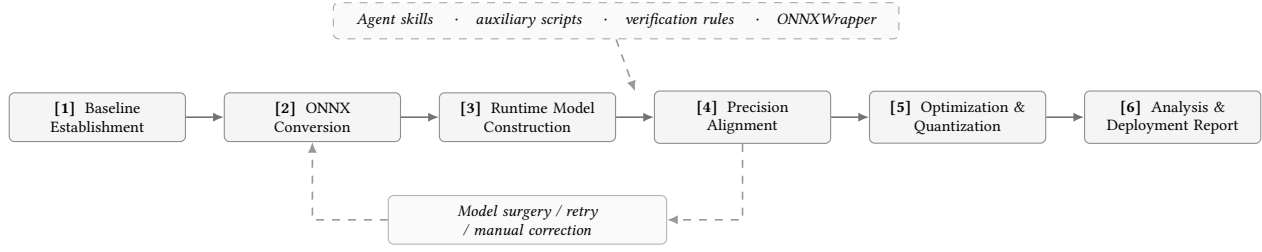


Fig. 2: The “skill + verification loop.” The six-stage pipeline is wrapped by injected domain knowledge (above) and a bounded repair path (below) that returns a failed verification to an earlier stage rather than terminating the workflow.

Additionally, the design philosophy differs across hardware runtime toolchains. TensorRT and RKNN conversion workflows typically do not rely on a common intermediate representation. They directly generate independent execution results for their respective hardware runtimes. In contrast, OpenVINO and QAI RT have greater potential to support multiple hardware platforms through shared intermediate formats. Different toolchains and hardware backends also differ significantly in quantization and tensor layout support. In this case, adaptation design must simultaneously address precision, quantization strategy, and data layout conversion characteristics.

Table 2 summarizes the settings and operations performed by each target runtime in the edge AI model deployment flow of Fig. 1.

The following sections detail each runtime’s specific deployment pipeline, agent skill specifics, and common repair actions.

It is important to note that the model quantization is performed directly or indirectly. The direct quantization is performed by the tools that use the **single-stage conversion** scheme as described in Section 3.1. In the direct model quantization scheme, quantization parameters are directly embedded in the conversion tool, with the output being the quantized model (RKNN via `do_quantization` parameter, TensorRT via `trtexec --int8` flag, QAI RT via QNN Converter quantization options). The **dual-stage conversion** scheme adopts the indirect model quantization. In this scheme, the model conversion is performed first, and then the converted model independently runs the quantization process (i.e., OpenVINO uses NNCF to quantize IR, ONNX Runtime QNN uses `onnxruntime.quantization` to convert to QDQ model). Moreover, quantization steps across runtimes can be divided into two categories: **post-conversion independent quantization** (first converting to device format, then quantizing that format) and **inline conversion quantization** (quantization parameters specified in the conversion tool, output is the quantized model).

The output formats of each runtime can be categorized as “device-agnostic” and “device-specific,” with most runtimes supporting both. Device-agnostic formats (e.g., OpenVINO IR, ONNX) indicate the format is not bound to a single hardware platform at the format level. It does not guarantee complete cross-device portability, where different devices have different operator support ranges, and some operators may only execute on specific devices. Device-specific formats (e.g., TensorRT Engine, RKNN `.rknn`, QAI RT Context Binary) are bound to specific hardware architectures and require recompilation for target devices.

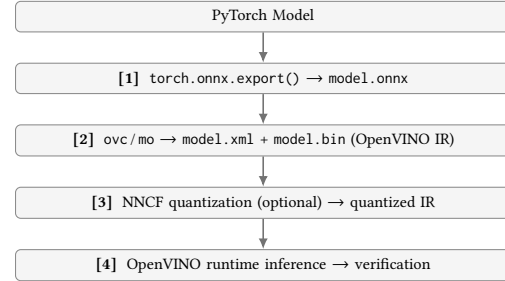


Fig. 3: OpenVINO deployment pipeline.

4.1 OpenVINO Deployment Pipeline

OpenVINO adopts the post-conversion independent quantization method. OpenVINO has three implementation methods. (1) Traditional step-by-step approach. The model is first converted to IR via `ovc/mo`, and then it runs NNCF quantization script. (2) Combined API method. It sequentially calls `convert_model()` and `nncf.quantize()` in the same script, with intermediate IR not saved to disk. (3) Integration packages are used for quantization, such as `optimum-intel` for single-line conversion and quantization. In contrast, RKNN, TensorRT, and QAI RT belong to the latter, with quantization parameters directly written in conversion commands (e.g., `trtexec --int8, rknn.config(do_quantization=True)`), output being the quantized model.

Agent skill details.

- **Export.** Agent uses OpenVINO-compatible settings (`opset version`, `dynamic axis handling`) to call `torch.onnx.export()`.
- **Conversion.** Agent uses `ovc` (OpenVINO Converter) with target device specification (CPU, GPU, and NPU).
- **Quantization.** Agent uses representative dataset for NNCF post-training quantization. Agent manages calibration data loading.
- **Verification.** Agent uses cosine similarity and element-wise tolerance to compare OpenVINO inference output with PyTorch reference output.

4.2 RKNN Deployment Pipeline

Agent skill details.

- **ONNX simplification.** Agent runs `onnx-simplifier` before RKNN conversion to clean the computational graph.

Table 2: Cross-runtime mapping of the common deployment stages.

Common Stage	OpenVINO	RKNN	TensorRT	QAIRT (QNN)	ONNX Runtime QNN
Model Source	PyTorch	PyTorch	PyTorch	PyTorch	PyTorch
ONNX Export	<code>torch.onnx.export()</code>	<code>torch.onnx.export()</code>	<code>torch.onnx.export()</code>	<code>torch.onnx.export()</code>	<code>torch.onnx.export()</code>
Runtime Conversion	ovc / mo $\rightarrow$ IR (.xml + .bin), optional model cache	RKNN-Toolkit2 $\rightarrow$.rknn	ONNX Parser $\rightarrow$ Network Definition $\rightarrow$ Builder $\rightarrow$ Engine/Plan (.engine)	QNN Path: QNN Converter $\rightarrow$ QNN Model, optional AOT compilation $\rightarrow$ Context Binary (.bin); SNPE Path: SNPE Converter $\rightarrow$ DLC (.dlc)	ONNX Runtime + QNN EP direct execution (runtime compilation), optional QNN context binary
Conversion Mode	Device-agnostic (IR) + device-specific (model cache)	Device-specific (.rknn)	Device-specific (Engine/Plan) + timing cache	QNN: device-specific (Context Binary) + device-agnostic (QNN Model / .so); SNPE: device-agnostic (DLC)	Device-agnostic (.onnx) + device-specific (QNN context binary)
Optimization & Quantization	NNCF PTQ: INT8/INT16 post-training quantization	do_quantization parameter, single-stage quantization	trtexec --int8 build-time quantization	QNN Converter built-in quantization or AIMET two-stage	onnxruntime.quantization converts to QDQ, then hands off to QNN EP
Verification	Cosine similarity, element-wise tolerance	Same as left	Same as left	Same as left	Same as left

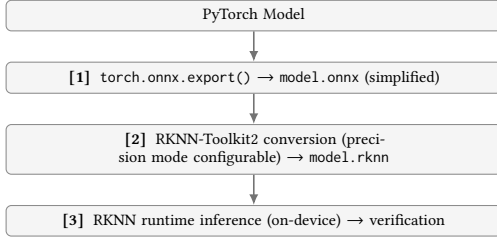


Fig. 4: RKNN deployment pipeline.

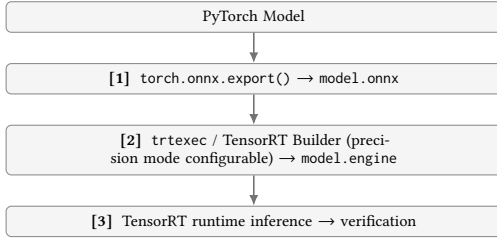


Fig. 5: TensorRT deployment pipeline.

- **Conversion and quantization.** Agent uses RKNN-Toolkit2 Python API, specifying platform configuration (rk3588) and do_quantization parameter. Conversion and mixed quantization are completed in the same tool; agent manages per-layer quantization configuration and calibration datasets.
- **Verification.** Agent performs on-device inference via RKNN runtime, and it compares output with reference values, accounting for quantization tolerance.

4.3 TensorRT Deployment Pipeline

This study adopts the **ONNX $\rightarrow$ TensorRT** path (the first deployment path described in Section 2.4), not the Torch-TensorRT path.

Agent skill details.

- **Export.** ONNX exports with TensorRT-compatible opset and explicit batch dimension.

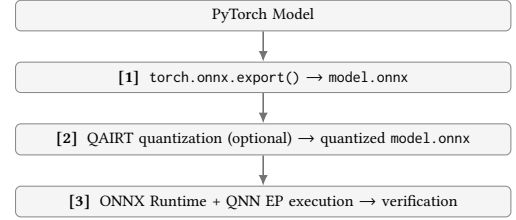


Fig. 6: ONNX Runtime QNN deployment pipeline.

- **Conversion and calibration.** Agent uses trtexec or Python TensorRT API, configuring builder settings (workspace size, FP16/INT8 flags, dynamic shape optimization profiles). Quantization parameters are specified at build time; agent manages calibration cache generation and reuse.
- **Verification.** Agent compares TensorRT output with PyTorch reference; handles numerical differences from reduced precision.

4.4 ONNX Runtime QNN Deployment Pipeline

ONNX Runtime supports multiple hardware backends through the Execution Provider architecture, including QNN EP (Qualcomm), OpenVINO EP (Intel), and TensorRT EP (NVIDIA). Each EP handles device-specific compilation and caching. For example, QNN EP provides `ep.context_enable` to serialize compilation results as QNN context binary, OpenVINO EP provides `cache_dir` for caching compiled models, and TensorRT EP provides engine cache mechanisms. This architecture enables ONNX Runtime to use the unified ONNX format as a device-agnostic intermediate representation while preserving device-specific optimization capabilities through each EP. **This study focuses on the QNN EP implementation**, with its deployment pipeline shown in Fig. 6.

ONNX Runtime QNN EP quantization belongs to the post-conversion independent quantization method. That is, QNN EP itself does not handle quantization and requires the input model to already be in QDQ format (ONNX model with QuantizeLinear/DequantizeLinear nodes). Quantization must

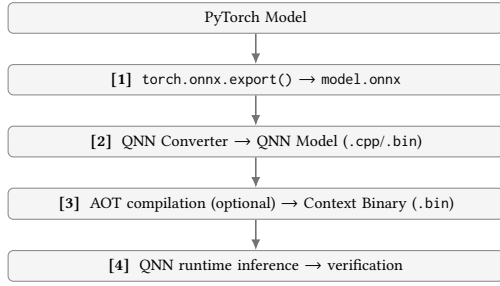


Fig. 7: QAI RT QNN deployment path, preferred for production.

be completed independently before QNN EP execution, using `onnxruntime.quantization` tools to convert floating-point models to QDQ format before loading by QNN EP.

Agent skill details.

- **Export.** ONNX exports with QNN EP-compatible opset version; agent must confirm the opset support range corresponding to the target QAI RT SDK version.
- **Quantization.** Optional use of QAI RT quantization toolchain for INT8/INT16 calibration on ONNX models. Agent manages calibration data loading and quantization parameter configuration.
- **Execution.** Agent uses ONNX Runtime with QNN EP for inference; agent configures QNN backend path, sets HTP execution mode (QNN_HTP, QNN_HTP_FP16), and device target (CPU/HTP).
- **Verification.** Agent compares QNN EP inference output with PyTorch reference output, and handles numerical differences from quantization and NPU precision limitations.

4.5 QAI RT Deployment Pipeline

QAI RT is the core target runtime of the original AIPC framework [16], supporting both QNN and SNPE deployment paths.

The QNN path, shown in Fig. 7, uses QNN Converter to convert ONNX models to QNN Model (.cpp/.bin), which can be further compiled via AOT compilation to generate Context Binary for accelerated loading. QNN Model is loaded and executed by QNN backend libraries (.so/.dll), supporting HTP (NPU), GPU, CPU, and other backends.

QAI RT supports two quantization modes. **Single-stage mode:** QNN Converter has built-in quantization parameters, completing quantization during conversion. **Two-stage mode:** It first uses AIMET (AI Model Efficiency Toolkit) for ONNX model quantization (supporting PTQ and QAT), and then passes the quantized model to QNN Converter for conversion. AIMET provides finer-grained quantization control, suitable for scenarios with high precision requirements.

In the SNPE path of Fig. 8, the agent uses SNPE Converter to convert ONNX models to DLC (Deep Learning Container) format. DLC is a device-agnostic format that can execute across different backends (CPU, GPU, DSP) on the same SoC, suitable for rapid verification and cross-backend testing. Additionally, SNPE can optionally enable model cache to generate device-specific cache files

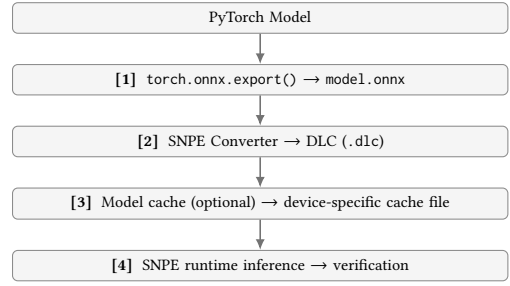


Fig. 8: QAI RT SNPE deployment path, used for rapid prototype verification.

for accelerated loading, but its cache parameters differ from QNN’s Context Binary. QNN Context Binary requires actively specifying `--soc_version` and other parameters for AOT compilation, while SNPE’s model cache is automatically managed by the runtime with fewer parameters.

Agent skill details.

- **Export.** ONNX export with QAI RT-compatible opset version.
- **Conversion (QNN).** Agent configures `qnn-onnx-converter` with `--preserve_io` to maintain I/O interface consistency.
- **Conversion (SNPE).** Agent configures SNPE Converter with quantization options.
- **Quantization.** Agent uses QAI RT quantization toolchain for INT8/INT16 calibration; agent manages calibration data loading and quantization parameter configuration.
- **Verification.** Agent compares QAI RT inference output with PyTorch reference output; handles numerical differences from quantization and NPU precision limitations.

4.6 ONNXWrapper Inference Replacement and Layout Adaptation

A recurring challenge in multi-runtime deployment is the mismatch between the data layout assumed by model code (typically PyTorch’s default NCHW) and the layout required by certain hardware backends, along with optimization needs. Manually inserting transpose operations for each model is tedious, error-prone, and complicates the agent’s repair logic.

For layout adaptation, ONNXWrapper, introduced in Section 3.1, analyzes the ONNX model layout and converts it as needed by the target runtime (e.g., between NCHW and NHWC), while performing shape consistency checks to verify conversion correctness. Because the Wrapper sits at the inference-replacement boundary, this adaptation requires no change to the original ONNX inference program logic.

QAI RT layout optimization. In QAI RT’s deployment implementation, ONNXWrapper’s layout adaptation is treated as part of the model optimization workflow. QNN models preserve layout options during conversion (layout preservation), and ONNXWrapper compares layout differences between the QNN backend and the original ONNX model, automatically performing layout conversion during the optimization stage. Notably, such layout conversion operations

are more efficiently executed on CPU than on NPU, so ONNXWrapper’s conversion operations are scheduled on the CPU side to avoid introducing additional computational overhead on the NPU.

RKNN layout adaptation. The RK3588 NPU’s compute units are optimized for NHWC tensor layout. When standard NCHW ONNX models are directly passed to RKNN-Toolkit2, layout mismatches occur and produce incorrect output results. ONNXWrapper automatically pre-converts the computational graph to NHWC before conversion, thereby eliminating layout-related failures. The ONNXWrapper method is integrated as a pre-conversion step into the agent’s RKNN deployment skill, automatically invoked before RKNN-Toolkit2 conversion, reducing the agent’s repair burden.

5 CASE STUDIES

This section evaluates the effectiveness of the agentic deployment method through two sets of comparative experiments: (1) automated deployment using AIPC agent skills and verification loops; (2) direct AI agent model conversion without the AIPC framework, serving as a baseline control. Both experiments use the same agent model (MIMO v2.5 / DeepSeek V4 Pro Preview) and target models, differing only in whether the agent is injected with structured deployment domain knowledge (operator compatibility tables, toolchain instructions, common failure patterns, and staged verification rules). The experiments in this section only verify deployment at NPU FP16 precision (non-TensorRT parts), without considering performance, throughput, or other optimization factors, focusing on the feasibility of the deployment workflow itself.

Field descriptions in the experimental tables are as follows.

- **End-to-end deployment time** includes agent thinking, code generation, toolchain execution, and verification. Due to differences in execution environments, LLM inference latency fluctuations, and varying numbers of AI steps, the same target’s end-to-end deployment time may drift across experiments; this work did not perform performance optimization on the deployment environment.
- **Repair actions** record specific repair measures taken by the agent when conversion fails.
- **Human intervention count** measures the number of steps requiring manual correction, serving as the core metric for evaluating automation degree, including confirming or installing additional libraries, humans discovering and correcting workflow errors, AI halting and waiting for human intervention to resume, and humans proactively intervening to force error correction.
- **Status** indicates deployment results (passed, partially passed, not completed).

5.1 Experimental Setup

This study selects two representative CV models as test targets: **Real-ESRGAN** [17] and **YOLO-World** [3]. Real-ESRGAN (Enhanced Super-Resolution Generative Adversarial Networks) is a CNN-based image super-resolution model with regular structure and simple operator types. It is primarily composed of convolutional and upsampling layers, making it suitable as a baseline test case for deployment workflows. YOLO-World is an open-vocabulary object detection model using a dual-encoder architecture (CLIP text

encoder and YOLO visual backbone). It requires multimodal inputs and more complex operator combinations. These two models represent “structurally regular” and “multi-encoder complex” model categories, helping observe performance differences of agent-based deployment across varying model complexities.

Experimental prerequisites. The following data should be prepared for each test model before the agent begins deployment to establish baselines. (1) Model inference script should be present to ensure that the converted model runs correctly as in the original framework (PyTorch). (2) Model weights and (3) test inputs are required to ensure that the model can be executed as in the original framework. (4) Reference outputs are needed to verify that the model’s output matches the expected results. All models must be confirmed to run in the original environment before entering the deployment workflow to ensure baseline correctness.

The agentic deployment workflow in this case study primarily uses **MIMO Code** (a CLI agent framework) for task orchestration and tool calls, driven by **MIMO Auto** mode. The deployment uses the never-ask settings (no user interruption for queries), without additional skill installation or other configuration adjustments. Internally, it may dynamically switch between MIMO Pro and Flash models. It should be noted that MIMO Code currently does not support Windows on ARM environments. For the QAIRT deployment target on Windows on ARM64 (Snapdragon X Plus), **Codex CLI** [10] with **DeepSeek V4 Pro Preview** is used as an alternative agent-driven solution.

The execution environments for this study are as follows.

- **TensorRT**: GTX 1070 Ti, Ubuntu 24.04 Docker.
- **OpenVINO**: Windows 11, Lunar Lake 228v, OpenVINO 2025.04.
- **RKNN**: Ubuntu 24.04 WSL, Orange Pi 5 (RK3588S, Ubuntu 22.04), remote deployment using SSH.
- **ONNX Runtime QNN**: Windows 11 ARM64, Snapdragon X Elite, QAIRT SDK 2.45, ONNX Runtime 1.22, onnxruntime_qnn 2.3.0, olive-ai 0.13.0.

5.2 Experiment 1: AIPC Agentic Deployment

This experiment group uses the AIPC framework, including complete agent skill modules (toolchain instructions, operator compatibility tables, common failure patterns) and staged verification loops. The experimental workflow is driven by the following three prompt instructions.

- **Inst. 1**: Create AIPC project in this folder. This instruction asks the agent to create an AIPC project in the specified folder.
- **Inst. 2**: [use provided env setting, fp16, <target>]. Auto fill the left fields and show the config for me to confirm. This instruction provides the environment settings (FP16 precision, target runtime), with the agent auto-filling remaining configuration and displaying for confirmation.
- **Inst. 3**: Do all the work. This instruction asks the agent to execute the complete deployment workflow and observe operation status.

This experiment group does not include QAIRT targets. As AIPC framework’s original validation runtime, QAIRT has been fully verified in prior research [16] to complete end-to-end deployment with no (or near-zero) human intervention. The goal of this experiment is to evaluate the feasibility of extending the AIPC methodology to other heterogeneous runtimes (OpenVINO, RKNN, TensorRT, ONNX Runtime QNN), thus focusing on these four new targets.

5.2.1 Real-ESRGAN Super-Resolution. Table 3 reports the results for Real-ESRGAN.

During ONNX Runtime QNN (HTP) execution, although we did not describe this error handling in the skill, the agent handled the error autonomously in two steps. First, Olive FP16 conversion created duplicate Cast output names, which violates the single static assignment (SSA) property of the graph; the agent renamed the duplicated node outputs. Second, the resulting nodes were no longer topologically sorted after the SSA fix, and the agent applied a manual topological sort.

5.2.2 YOLO-World (Open-Vocabulary Detection). Table 4 reports the results for YOLO-World.

During OpenVINO environment deployment, the agent attempted to use onnxsim to simplify the ONNX computational graph, but the simplified model failed at the conversion stage. The agent identified that the CLIP text encoder’s complex structure might exceed onnxsim’s processing capability, so it abandoned the simplification step and used the unsimplified ONNX model for direct conversion, ultimately completing deployment successfully. This observation highlights the importance of pre-configuring the deployment environment: if toolchain versions, dependency libraries, and target device drivers are pre-installed, the agent’s exploration time and failure risk during environment preparation can be reduced.

During RKNN environment deployment, after completing environment detection, the agent autonomously decided to upgrade the PyTorch version and completed inference verification through the RKNN CPU simulator. However, the agent treated simulator verification as a completion signal, pausing the workflow and asking the user whether to proceed with the next step of remote device deployment. After manual confirmation, the agent continued to complete RK3588 on-device deployment. The final deployment passed verification, but analysis showed four operators fell back to CPU execution, reflecting that YOLO-World’s complex operator combinations still have compatibility limitations on RKNN.

During TensorRT environment deployment, the agent exhibited two deviations from skill instructions: (1) the agent once misjudged that the target hardware did not support FP16 precision, autonomously switching to FP32 mode, then recovering FP16 settings after re-confirmation; (2) the agent skipped the ONNXWrapper layout adaptation step specified in the AIPC skill module, directly executing TensorRT conversion. These two deviation behaviors indicate that even within the structured skill injection framework, agents may still deviate from the predetermined workflow due to reasoning biases, emphasizing the necessity of verification mechanisms and execution harnesses during deployment. Ultimately, the agent successfully completed deployment in FP16 mode after correction.

In the ONNX Runtime QNN (HTP) environment, two tests were conducted, both facing the same core challenge: the Einsum operator in the YOLO-World v2 model is not supported by QNN EP, causing the computational graph to be split into six independent EPContext nodes (rather than the expected single node). In the first test, the agent attempted multiple times to generate QNN context binary but all failed, ultimately choosing to skip this optimization step and execute inference with the split computational graph; however, the AIPC skill module actually already contained handling procedures for such situations, and the agent failed to correctly trigger that repair path. In the second test, the agent attempted equivalent replacement repair for the Einsum operator, replacing the Einsum node with an equivalent matrix multiplication subgraph, ultimately completing deployment and passing verification.

5.3 Experiment 2: Direct AI Conversion Without Structured Domain Knowledge (Baseline Control)

This experiment group does not use the AIPC framework’s skill modules and verification loops; the AI agent directly attempts PyTorch to target runtime conversion using only its general model knowledge. Compared to Experiment 1, the agent is not injected with operator compatibility tables, toolchain instructions, or common failure patterns. The purpose of this experiment is to quantify the impact of structured domain knowledge injection on agent deployment capability.

The test method is as follows. Given a PyTorch model and inference script, we use the prompt “please port the PyTorch PROJECT using {framework},” with hints provided if environments are already set up.

5.3.1 Real-ESRGAN Super-Resolution. Table 5 reports the baseline results for Real-ESRGAN.

In this experiment, for TensorRT and OpenVINO NPU the agent could directly understand the workflow and generate usable models. For ONNX Runtime QNN, the agent used the older onnxruntime-qnn 1.x version, successfully running Real-ESRGAN inference via QNN EP on HTP. The agent identified the QNN EP limitation requiring fixed input dimensions for NPU targets during the process and adjusted model configuration accordingly.

QAIRT QNN conversion required additional analysis of the SDK directory structure, converting the model to the SNPE workflow and generating context binary, taking approximately one hour. RKNN (RK3588) encountered output similarity mismatch with PyTorch reference values during inference; the agent attempted modifying operators and INT8 quantization configuration but could not complete, stopping after one hour and reporting incomplete. Subsequent testing found that if manually prompted that the hardware is NHWC format, RKNN conversion could pass.

Additionally, in the OpenVINO experiment, we discovered that on one occasion the AI generated test images itself, ran through the workflow, then was asked to use built-in test images, confirming functionality was correct.

Notably, in the ONNX Runtime QNN test, the agent autonomously selected the deprecated onnxruntime-qnn 1.x version to complete conversion; subsequent manual switching to the

Table 3: Experiment 1 (with AIPC skill): Real-ESRGAN super-resolution.

Target	End-to-End Time	Repair Actions	Human Intervention	Status
OpenVINO (NPU)	18 min	None	0	✓Passed
RKNN (RK3588)	16 min	None	0	✓Passed
TensorRT (FP16)	21 min	None	0	✓Passed
ONNX Runtime QNN (HTP)	26 min	Olive optimization error, rename operator	0	✓Passed

Table 4: Experiment 1 (with AIPC skill): YOLO-World open-vocabulary detection.

Target	End-to-End Time	Repair Actions	Human Intervention	Status
OpenVINO (NPU)	12 min	Abandoned onnxsim, adopted direct conversion path	1. Installed onnxsim dependency (later abandoned due to conversion failure)	✓Passed
RKNN (RK3588)	25 min	None	1. Paused after environment detection, waiting for user confirmation of remote deployment	✓Passed
TensorRT (FP16)	17 min	None	1. Agent did not strictly follow skill instructions, skipped ONNXWrapper step	✓Passed
ONNX Runtime QNN (HTP)	21 min / 22 min	Two tests: context binary generation fail (skip) / repaired Einsum operator	0	✓Partially passed, acceptable

Table 5: Experiment 2 (without skill): Real-ESRGAN super-resolution.

Target	End-to-End Time	Repair Actions	Human Intervention	Status
OpenVINO (NPU)	~19 min	None	0	✓Passed
RKNN (RK3588)	~90 min	Reshape operator repair, INT8 quantization adjustment, CPU simulator	0, then prompted input conversion to NHWC; can complete but takes ~2 h	✗Not completed
TensorRT (FP16)	~20 min	None	0	✓Passed
ONNX Runtime QNN (HTP)	20 min	Fixed shape	0	✓Passed (using older onnxruntime-qnn 1.x)
QAIRT QNN (FP16)	~60 min	SDK structure analysis, SNPE workflow conversion	0	✗Below expectations: converted from QNN to SNPE workflow

2.x version caused failures. Considering the agent successfully completed end-to-end inference under the 1.x version, this test result was still marked as **usable (passed)**. This observation also suggests that without skill explicitly specifying package versions, agents may default to non-current stable versions, indicating a need to annotate recommended version ranges in skill modules.

Additionally, without skill, we observed that the agent sometimes tends to traverse the entire `onnxruntime-qnn` installation directory to locate required libraries or configuration files, rather than directly accessing them along predetermined paths. This extensive scanning behavior introduces a large amount of unnecessary directory structure information, causing significant additional input token consumption and correspondingly longer response times. In contrast, skill, by providing explicit toolchain paths and API references, may effectively suppress such inefficient exploration behavior, reducing the agent’s token cost and execution latency.

5.3.2 YOLO-World (Open-Vocabulary Detection). Table 6 reports the baseline results for YOLO-World.

In this experiment, OpenVINO and TensorRT both passed; RKNN conversion previously required manual prompting that the hardware is NHWC format, and QAIRT testing had severe drift, so this section was not re-executed.

In the ONNX Runtime QNN experiment, we specified execution in a particular Python virtual environment (venv) and explicitly

instructed the agent to perform inference in QNN Context Binary mode. The agent stopped after attempting to execute Context Binary and failing; the user then issued instructions in non-Context Binary mode, and the agent executed as instructed and reported deployment success. However, actual verification revealed that the entire inference process did not use NPU/HTP, and hardware monitoring tools confirmed NPU was inactive throughout.

After feeding back the “NPU inactivity” observation to the agent, the agent then identified the root cause and made corrections, ultimately completing inference correctly executed on the HTP backend. This case illustrates that even when the agent completes inference and reports success according to user instructions, independent hardware activity verification is still needed to confirm whether the target accelerator was actually activated, one cannot judge deployment effectiveness solely based on the agent’s report.

After the test failure, we attempted to reopen the conversation with the Claude Sonnet 4.6 model (new conversation), letting the agent read artifacts and work files left from the previous conversation as context, to re-attempt YOLO-World’s ONNX Runtime QNN Context Binary generation. This time, Context Binary generation and inference verification were successfully completed in the new conversation.

Table 6: Experiment 2 (without skill): YOLO-World open-vocabulary detection.

Target	End-to-End Time	Repair Actions	Human Intervention	Status
OpenVINO (NPU)	20 min	Dynamic shape to fixed shape	0	✓Passed
RKNN (RK3588)	Not executed	Not executed	Not executed	Not executed
TensorRT (FP16)	18 min	Dynamic shape to fixed shape	0	✓Passed
ONNX Runtime QNN (HTP)	~60 min	—	2. Tested using non-context-binary mode, reported NPU inactivity	✗Not passed
QAIIR QNN (FP16)	Not executed	Not executed	Not executed	Not executed

Table 7: Summary of both experiment groups.

Experiment Group	Success Rate	Test Time (Successful)	Human Interv.
Exp. 1 (with AIPC skill)	8/8 (100%)	Avg. 19.5 min	3
Exp. 2 (without skill)	3/5 (60%)	Avg. 19.8 min	2

5.4 Analysis

Table 7 summarizes the observation metrics from both experiment groups. It should be noted that this study’s experimental sample size is limited (4–8 deployment tests per group), and the following data are preliminary observations only, not precise statistical analysis, lacking statistical significance.

“Deployment success rate” is the number of passed verification deployments divided by total attempts (excluding unexecuted targets); “deployment test time” calculates only the average time of successful deployments, excluding failed cases’ times (as their duration is unrelated to deployment feasibility); “human intervention count” includes all human intervention events such as confirming environments, installing dependencies, and suggesting correction directions.

From this study’s deployment results, for TensorRT and OpenVINO, AI agents already demonstrate considerable understanding of common CV model deployment tasks: they can handle ONNX conversion, dynamic shape profiles, and basic deployment-time repairs, completing basic inference pipeline construction. RKNN’s layout requirements are more obscure, making it difficult for AI agents to fully master all conversion conditions; in the cases, it was only after manual explicit prompting to convert from NCHW to NHWC format that RKNN conversion could pass. ONNX Runtime QNN benefits from the broad support of the standardized ONNX format, with agents demonstrating considerable capability in basic conversion, but facing challenges of limited QNN EP operator coverage (e.g., not supporting Einsum) and Olive optimization tool integration, requiring agents to handle operator equivalent substitution and context binary generation failures.

Synthesizing the above observations, the common problem across runtimes is that the agent’s failure modes are highly correlated with its command of deployment knowledge. When operator support boundaries, layout requirements, and toolchain behavior can be explicitly recorded and injected into the agent’s context (as provided by AIPC skill modules), the agent is better able to follow correct repair paths; conversely, when this knowledge

is scattered in documentation or implicit in toolchain behavior, agents tend to deviate from workflows or adopt inefficient trial strategies.

Synthesizing observations from both experiments, a more fundamental issue emerges: **AI agents in engineering tasks exhibit a universal tendency toward drift and hallucination.** Whether or not AIPC skill modules are used, agents exhibit varying degrees of deviation behavior during deployment across multiple runtimes, misjudging hardware capabilities (e.g., misjudging FP16 support in TensorRT environments), skipping critical steps (e.g., omitting ONNXWrapper layout adaptation), and autonomously modifying environment configurations (e.g., upgrading PyTorch version without authorization in RKNN environments). Notably, **the longer the deployment workflow steps and the more the operations deviate from conventional paths** (e.g., AIPC framework requires layout adaptation via ONNXWrapper rather than directly calling hardware runtime APIs), the more severe the agent’s deviation becomes. The YOLO-World deployment cases clearly demonstrate this: compared to the structurally regular Real-ESRGAN, YOLO-World triggered agent deviation behavior across multiple runtimes, even under AIPC skill injection constraints that could not be fully avoided.

These observations indicate that AI agents in engineering practice should be paired with appropriate verification mechanisms and staged checks, rather than relying entirely on the agent’s autonomous decision-making. The AIPC framework demonstrated in this study serves as a preliminary porting verification tool; its “skill + verification loop” mechanism can constrain agent deviation behavior to some extent, but still has shortcomings when facing complex models or long-workflow tasks. In practical engineering deployment, more rigorous staged verification, output comparison, and anomaly detection mechanisms are needed to ensure each step of the agent executes on the expected trajectory.

5.5 Observations and Limitations

This section synthesizes experimental observations and methodological limitations, explaining the current research coverage and unresolved issues.

Model complexity coverage boundary. Current experiments only cover two CV models: Real-ESRGAN and YOLO-World. Structurally regular convolutional networks (like Real-ESRGAN) have higher conversion success rates across runtimes with fewer agent deviation behaviors; models involving multi-encoder architectures (like YOLO-World) can still complete deployment on most runtimes, but agent deviation behaviors are significantly more frequent (e.g., skipping ONNXWrapper, misjudging hardware capabilities), and

operator fallback to CPU occurs on RKNN, reflecting that complex model structures increase the difficulty of agents correctly following deployment workflows. Transformer, LLM, and other more complex architectures’ complete performance across runtimes has not yet been systematically verified.

Cross-platform commonality and differences of operator repair not fully clarified. In handling unsupported operators, runtimes actually share multiple repair mechanisms, operator substitution, CPU fallback, and plugin operator strategies are all used in OpenVINO, RKNN, TensorRT, and QNN EP, with differences primarily manifesting in trigger conditions, substitution strategies, and default behaviors of each mechanism. Current experiments are case-by-case observations, lacking systematic cross-platform comparison, and the applicable boundaries and best practices of each repair mechanism across different runtimes cannot yet be generalized.

Quantization and optimization not yet implemented. This research currently focuses on deployment feasibility verification and has not yet implemented quantization and optimization testing.

Agent model generalization boundary. This study primarily uses MIMO v2.5 as the agent-driven model, only switching to Codex CLI with DeepSeek V4 Pro Preview in the Windows on ARM environment (where MIMO Code does not yet support the platform). Different agent models may have significant differences in skill compliance, failure recovery capability, and platform knowledge generalization, requiring larger-scale cross-model verification.

6 CONCLUSION

This paper extends the AIPC agentic deployment methodology to multi-runtime scenarios, including Intel OpenVINO, Rockchip RKNN, NVIDIA TensorRT, and ONNX Runtime QNN. This method targets single-model-to-single-runtime pairings for automated deployment. Preliminary results demonstrate the “skill + verification loop” design pattern’s feasibility across different toolchains, where LLM agents help with model conversion, operator repair, and precision verification under structured knowledge injection. The ONNXWrapper layer removes tensor layout adaptation from the agent’s manual repair burden. We have conducted the controlled comparison between deployments performed with and without structured knowledge injection. Our results show that the agent’s failure modes can be attributed to knowledge availability rather than to model capability alone.

For the structurally regular vision models we tested, agents can successfully complete deployment on most runtimes. On the other hand, general knowledge alone is insufficient for reliable automated handling when model structures become more complex (e.g., multi-encoder architectures). Examples of the complex situations include: target runtime operator coverage is limited (e.g., ONNX Runtime QNN not supporting Einsum), or runtimes have implicit tensor layout constraints (e.g., RKNN requiring NHWC format) in less common or complex scenarios.

From a more fundamental perspective, hardware-specific inference runtimes are inevitable on many edge platforms. Compared to generic PyTorch execution, using native hardware runtimes (such as NPU and GPU dedicated inference engines) typically brings significant performance improvements and power consumption

optimization. This is especially critical for resource-constrained edge devices. In the past, deploying models to these underlying hardware required developers to deeply understand each platform’s toolchain, operator support range, quantization schemes, and compilation configuration, a tedious and highly barrier-laden process. As AI agent technology matures, this gap is being bridged: agents can understand, orchestrate, and execute these complex hardware adaptation workflows, making what was previously hardware deployment work achievable at lower cost and in shorter time.

REFERENCES

- [1] Ascend Community. 2026. Agent Skills: AI Agent Toolkit for Huawei Ascend NPU. <https://github.com/Ascend/agent-skills>. Accessed: 2026-09-03.
- [2] Terry Chen, Zhifan Ye, Bing Xu, et al. 2026. AVO: Agentic Variation Operators for Autonomous Evolutionary Search. *arXiv preprint arXiv:2603.24517* (2026).
- [3] Tao Cheng, Lin Song, Yixiao Ge, Wenyu Liu, Xinggang Wang, and Ying Shan. 2024. YOLO-World: Real-Time Open-Vocabulary Object Detection. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*.
- [4] Mengling Deng, Yuanpeng Chen, Sheng Yang, et al. 2026. EdgeFM: Efficient Edge Inference for Vision-Language Models. *arXiv preprint arXiv:2604.27476* (2026).
- [5] Intel Corporation. 2026. OpenVINO Toolkit: Open Visual Inference and Neural Network Optimization. <https://docs.openvino.ai/>. Accessed: 2026-09-03.
- [6] Jiajie Li, Erwei Wang, Zhiru Zhang, and Samuel Bayliss. 2026. From Human Guidance to Autonomy: Agent Skill System for End-to-End LLM Deployment on Spatial NPUs. *arXiv preprint arXiv:2606.07586* (2026).
- [7] Linux Foundation. 2026. ONNX: Open Neural Network Exchange. <https://onnx.ai/>. Accessed: 2026-09-03.
- [8] Microsoft Corporation. 2026. ONNX Runtime: Cross-Platform, High Performance ML Inference and Training Accelerator. <https://onnxruntime.ai/>. Accessed: 2026-09-03.
- [9] NVIDIA Corporation. 2026. NVIDIA TensorRT: Programmable Inference Accelerator. <https://developer.nvidia.com/tensorrt>. Accessed: 2026-09-03.
- [10] OpenAI. 2026. Codex CLI: Lightweight Coding Agent in the Terminal. <https://github.com/openai/codex>. Accessed: 2026-09-03.
- [11] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. 2019. PyTorch: An Imperative Style, High-Performance Deep Learning Library. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- [12] Qualcomm Technologies Inc. 2026. Qualcomm AI Engine Direct (QNN) SDK. <https://www.qualcomm.com/developer/software/qualcomm-ai-engine-direct>. Accessed: 2026-09-03.
- [13] Rockchip Electronics. 2026. RKNPU2: Rockchip Neural Processing Unit SDK. <https://github.com/rockchip-linux/rknpu2>. Accessed: 2026-09-03.
- [14] Taras Sereda, Burak Bartan, Ankita Nayak, et al. 2026. KForge: LLM-Driven Cross-Platform Kernel Generation for AI Accelerators. *arXiv preprint arXiv:2606.02963* (2026).
- [15] Marcin Spoczynski, Daniel Fleischer, Moshe Berchansky, et al. 2026. Xe-Forge: Multi-Stage LLM-Powered Kernel Optimization for Intel GPU. *arXiv preprint arXiv:2605.26118* (2026).
- [16] Jianhao Su, Zhanwei Wu, ShengTing Huang, and Weidong Feng. 2026. AIPC: Agent-Based Automation for AI Model Deployment with Qualcomm AI Runtime. *arXiv preprint arXiv:2604.14661* (2026).
- [17] Xintao Wang, Liangbin Xie, Chao Dong, and Ying Shan. 2021. Real-ESRGAN: Training Real-World Blind Super-Resolution with Pure Synthetic Data. In *Proceedings of the IEEE/CVF International Conference on Computer Vision Workshops (ICCVW)*.
- [18] Genghan Zhang, Shaowei Zhu, Anjiang Wei, et al. 2026. AccelOpt: A Self-Improving LLM Agentic System for AI Accelerator Kernel Optimization. *arXiv preprint arXiv:2511.15915* (2026).